

LAPORAN PRAKTIKUM

TEKNOLOGI IoT 2

“Komunikasi Asinkron dan WebSockets”



DISUSUN OLEH

Nama : Naurah Miftah Frizkiyah

Kelas : TRI 3B

NPM: 25782049

DOSEN :

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom.,

Andi Saptono, A.Md

PROGRAM STUDI TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2026

Latihan 1

Teori Jaringan (Keunggulan WebSockets vs Meta Refresh HTTP)

Teknologi koneksi WebSockets jauh lebih efisien dalam penggunaan data dan *bandwidth* internet dibandingkan metode *Meta Refresh HTTP* karena WebSockets membangun saluran komunikasi dua arah (*full-duplex*) secara persisten (*persistent TCP connection*). Setelah *handshake* awal berhasil dilakukan, server NodeMCU dan *browser* dapat saling berkiriman data secara *real-time* tanpa perlu mengulang header request HTTP yang berukuran besar pada setiap transaksi data. Sebaliknya, metode *Meta Refresh* mewajibkan *browser* untuk mengunduh ulang seluruh kode HTML, CSS, dan JavaScript secara berkala, yang menyebabkan konsumsi data membengkak secara signifikan. Selain itu, WebSockets tidak menyebabkan layar *browser* berkedip (*flicker*) karena pembaruan tampilan dilakukan secara parsial hanya pada elemen DOM tertentu yang berubah menggunakan JavaScript, tanpa perlu merusak dan memuat ulang (*reload*) seluruh halaman web dari awal.

Latihan 2

Analisis Waktu (Dampak Penggunaan delay(3000) pada WebSockets)

Penggunaan sintaks `delay(3000)` secara nekat di dalam fungsi `void loop()` akan menimbulkan bahaya sistematis yang fatal pada arsitektur komunikasi WebSockets. Fungsi `delay()` bersifat *blocking*, yang berarti CPU NodeMCU akan menghentikan seluruh eksekusi program dan mengabaikan semua proses latar belakang selama 3 detik penuh. Akibatnya, server AsyncWebSocket tidak dapat memproses *handshake*, mendengarkan paket incoming dari *browser*, maupun merespon *ping/pong frame* yang digunakan untuk menjaga koneksi tetap hidup (*keep-alive*). Kondisi ini menyebabkan komunikasi menjadi sangat lambat (*lagging*), perintah dari *browser* tertahan atau hilang, dan pada akhirnya sistem akan mengalami *WebSocket timeout* yang berujung pada terputusnya koneksi (*disconnect*) secara berulang.

Latihan 3

Modifikasi Front-End (Identifikasi ID Elemen HTML Target)

Dalam perancangan antarmuka *front-end*, pembaharuan tampilan dilakukan secara dinamis melalui teknik manipulasi DOM (*Document Object Model*) menggunakan JavaScript. Pendekatan ini memungkinkan *browser* mengubah data yang tampil di layar secara instan begitu pesan dari NodeMCU diterima, tanpa harus memuat ulang (*reload*) seluruh halaman web.

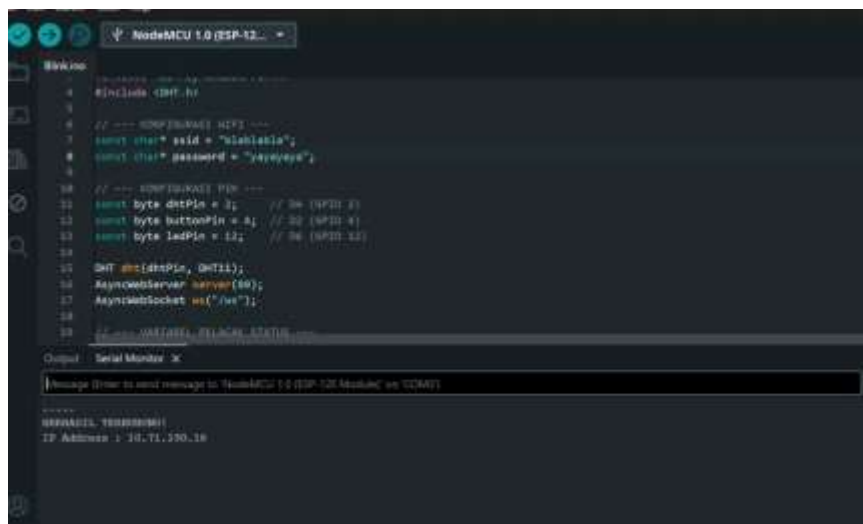
Proses pembaruan nilai tersebut bekerja dengan menyasar beberapa ID unik pada elemen HTML (`<span id="...">`), di antaranya:

- **ledState**: Berfungsi sebagai wadah teks untuk menampilkan status operasional LED saat ini (misalnya indikator "ON" atau "OFF").
- **temp** (atau **temperature**): Menjadi target injeksi data angka hasil pembacaan suhu lingkungan yang dikirimkan oleh sensor DHT.
- **connStatus** (atau **wsStatus**): Digunakan untuk menginformasikan status tautan komunikasi WebSocket antara *browser* dan NodeMCU secara *real-time* (seperti status "Connected" atau "Disconnected").

Latihan 4

Pengembangan Telemetry Sensor (Modifikasi Program Pembacaan Kelembapan)

Untuk mengintegrasikan telemetry pembacaan kelembapan (*humidity*) dari sensor DHT pada Praktikum 1, modifikasi program dilakukan pada sisi *back-end* C++ maupun *front-end* HTML/JavaScript. Pada program C++ NodeMCU, variabel baru ditambahkan untuk membaca data kelembapan menggunakan fungsi `dht.readHumidity()`. Data kelembapan tersebut kemudian digabungkan ke dalam format JSON pada fungsi `notifyClients()`, misalnya dengan menambahkan *key-value* `"hum\":\"" + String(currentHum) + "\"`. Pada sisi *front-end*, ditambahkan elemen tampilan baru berupa `<span id="humidity">--</span> %` di dalam kode HTML. Skrip JavaScript pada *browser* selanjutnya dimodifikasi untuk mengekstrak data `hum` dari pesan JSON yang diterima via WebSocket, lalu menginjeksikannya langsung ke elemen ID `humidity` menggunakan perintah `document.getElementById('humidity').innerHTML = JSONvar.hum` agar nilai kelembapan terkonfirmasi secara *real-time*.



```
NodeMCU 1.0 (ESP-12)
#include <DHT.h>

// --- CONFIGURASI WIFI ---
const char* ssid = "blablabla";
const char* password = "yayayaya";

// --- CONFIGURASI PIN ---
const byte dhtPin = 3; // D+ (SPD1 3)
const byte buttonPin = 5; // D- (SPD1 4)
const byte ledPin = 12; // D- (SPD1 12)

DHT dht(dhtPin, DHT11);
AsyncWebServer server(80);
AsyncWebSocket ws("/ws");

// --- JARANG, PELAKSI STATUS ---

Output: Serial Monitor
Message (Error to send message to "NodeMCU 1.0 (ESP-12 Module)" via "COM4")
SERIAL: 10/11/2020
IP Address: 10.71.190.16
```

